PROJECT REPORT

BitTrace

Bitcoin Transaction Forensics and Investigation System

MSc Cyber Security

Dipangshu Dey

2026

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
BTC	Bitcoin
CSV	Comma Separated Values
DB	Database
FK / PK	Foreign Key / Primary Key
GUI	Graphical User Interface
HTTP(S)	Hypertext Transfer Protocol (Secure)
JSON	JavaScript Object Notation
LLM	Large Language Model
OFAC	Office of Foreign Assets Control
P2PKH	Pay to Public Key Hash
P2SH	Pay to Script Hash
PDF	Portable Document Format
PNG	Portable Network Graphics
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDN	Specially Designated Nationals (list)
SQL	Structured Query Language
TXID	Transaction Identifier
UI	User Interface
UTC	Coordinated Universal Time
UTXO	Unspent Transaction Output

LIST OF TABLES

Table 1: Forensic concepts and the modules that use them....3
Table 2: Tools and technologies used in BitTrace....4
Table 3: Functional requirements....5
Table 4: Main source files and their responsibilities....6
Table 5: Database tables and their purpose....9
Table 6: Key settings in config.py....10
Table 7: Pattern detection rules....15
Table 8: Bundled threat lists....16
Table 9: Risk signals and their weights....16
Table 10: Sections of the PDF case report....18
Table 11: Pages of the BitTrace dashboard....19
Table 12: Automated tests in the tests/ folder (executed)....22
Table 13: Records created by the demo investigation (executed)....23
Table 14: Executed test cases....23
Table 15: Items not executed....25

LIST OF FIGURES

Figure 1: Overall system architecture of BitTrace....6
Figure 2: Data-processing flow between the main modules....8
Figure 3: Database structure of BitTrace (SQLite)....9
Figure 4: Investigation workflow of BitTrace....11
Figure 5: Request, retry and fallback logic of DataFetcher._get()....14
Figure 6: Decision flow of the risk engine....17
Figure 7: Question-answering flow of the AI assistant....18
Figure 8: Title block, summary and risk assessment of the PDF report generated for the demo investigation....21
Figure 9: Graph focus view and most central addresses in the PDF report....21

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION..... 1
1.1 Introduction and Background..... 1
1.2 Problem Statement and Motivation..... 1
1.3 Aim and Objectives..... 1
1.4 Scope of the Project..... 2
CHAPTER 2 BACKGROUND AND TECHNOLOGIES USED..... 3
2.1 How Bitcoin Transactions Work..... 3
2.2 Forensic Concepts Used in BitTrace..... 3
2.3 Tools and Technologies Used..... 4
CHAPTER 3 SYSTEM REQUIREMENTS..... 5
3.1 Functional Requirements..... 5
3.2 Non-Functional Requirements..... 5
3.3 Software and Hardware Environment..... 5
CHAPTER 4 SYSTEM ARCHITECTURE AND DESIGN..... 6
4.1 Overall Architecture..... 6
4.2 Module Overview..... 6
4.3 Data Flow..... 7
4.4 Database Design..... 8
4.5 Configuration Design..... 10
CHAPTER 5 METHODOLOGY AND INVESTIGATION WORKFLOW..... 11
5.1 Overall Workflow..... 11
5.2 Step-by-Step Explanation..... 11
5.3 Live Mode and Demo Mode..... 12
5.4 Hop Expansion..... 12
CHAPTER 6 SYSTEM IMPLEMENTATION..... 13
6.1 Input Validation (core/validation.py)..... 13
6.2 Blockchain Data Retrieval (core/data_fetcher.py)..... 13
6.3 Address Clustering (core/clustering.py)..... 14
6.4 Flow Graph (core/graph_builder.py)..... 15
6.5 Pattern Detection (core/pattern_detector.py)..... 15
6.6 Graph Analytics (core/graph_analytics.py)..... 15
6.7 Threat Intelligence and Entity Attribution..... 15
6.8 Risk Scoring (core/risk_engine.py)..... 16
6.9 Data Storage (db/database.py)..... 17
6.10 PDF Report Generator (core/report_generator.py)..... 17
6.11 Local AI Assistant (ai/ package)..... 18
CHAPTER 7 APPLICATION INTERFACE AND FEATURES..... 19
7.1 Investigate Page..... 19
7.2 Graph Explorer..... 19
7.3 Timeline, History and Statistics..... 20
7.4 AI Assistant Page..... 20
7.5 Exported PDF Report..... 20

CHAPTER 8 TESTING AND RESULTS..... 22
8.1 Testing Approach..... 22
8.2 Project Test Suite..... 22
8.3 End-to-End Checks on the Demo Investigation..... 23
8.4 Discussion of Results..... 24
8.5 Features Not Tested in This Environment..... 25
CHAPTER 9 SECURITY CONSIDERATIONS AND LIMITATIONS..... 26
9.1 Security Measures in the Implementation..... 26
9.2 Privacy and Operational Considerations..... 26
9.3 Limitations..... 26
CHAPTER 10 CONCLUSION AND FUTURE IMPROVEMENTS..... 28
10.1 Conclusion..... 28
10.2 Future Improvements..... 28

CHAPTER 1 INTRODUCTION

1.1 Introduction and Background

Bitcoin is a public, pseudonymous payment network. Every transaction ever made is written to a shared ledger called the blockchain, and anyone can read it. The ledger does not store names. It stores addresses, which are long strings of letters and numbers, and the amounts that moved between them. This makes Bitcoin transparent but not anonymous, because a careful investigator can follow the movement of funds from one address to the next and sometimes connect several addresses to the same owner.

This property is useful in cyber crime investigations. Ransomware payments, sanctions evasion and darknet market sales are often paid in Bitcoin, and the payments stay visible on the ledger forever. The difficulty is the size of the data. A single busy address can be part of thousands of transactions, and criminals deliberately split, chain and mix their funds to make the trail harder to read. Doing this by hand in an ordinary block explorer is slow and easy to get wrong.

BitTrace is the system built in this project to help with that work. Its full title is BitTrace: Bitcoin Transaction Forensics and Investigation System. The investigator enters one Bitcoin address, the system downloads the surrounding transactions, and the rest of the analysis is automatic. It groups addresses that probably belong to the same owner, draws the fund-flow graph, looks for known laundering patterns, checks addresses against published threat lists and gives every address a risk score with a written explanation. The result is saved in a local database and can be exported as a PDF case report.

1.2 Problem Statement and Motivation

The project started from a few practical problems that appear when a Bitcoin address has to be investigated:

- **Raw data is hard to read.** A block explorer shows one transaction at a time. It does not show the bigger picture of how funds moved across many addresses.
- **Ownership is hidden.** One person usually controls many addresses, but the ledger does not say which ones. Without grouping them, the investigator under-counts what a person received or sent.
- **Laundering patterns are easy to miss.** Peel chains, fan-out hubs, CoinJoin-style mixing and circular flows are visible only when the whole graph is examined.
- **Scores must be explainable.** In a forensic setting, a number such as 82 means little unless the reader can see why it was given. A black-box score cannot be defended in a report or in a viva.
- **Evidence has to be written down.** The findings must be saved and exported in a form that can be attached to a case file.

BitTrace was designed to address these problems with a single local tool that anyone with Python can run on a laptop.

1.3 Aim and Objectives

Aim: to design and build a Bitcoin transaction forensics system that traces an address across the transaction graph, clusters related addresses, detects common laundering patterns and produces an explainable risk score with an exportable evidence report.

The objectives, as implemented in the source code, are:

1. Retrieve real transaction history for a Bitcoin address from public blockchain APIs, with retries and a fallback server, and also support a built-in synthetic demo mode that works offline (`core/data_fetcher.py`, `core/demo_data.py`).
2. Group addresses that are probably controlled by one owner using the common-input-ownership rule and a change-address heuristic (`core/clustering.py`).
3. Build a directed fund-flow graph between addresses and compute graph measures such as PageRank and betweenness centrality (`core/graph_builder.py`, `core/graph_analytics.py`).
4. Detect five laundering-related patterns: peel chains, fan-out, fan-in, CoinJoin-style transactions and circular flows (`core/pattern_detector.py`).
5. Match addresses against bundled threat-intelligence lists and infer an entity type from behaviour (`core/threat_intel.py`, `core/entity_attribution.py`).
6. Give every address a 0 to 100 risk score with an itemised breakdown (`core/risk_engine.py`).
7. Store every investigation in a local SQLite database (`db/database.py`, `db/schema.sql`).
8. Present the work in a multi-page Streamlit dashboard with a graph explorer, timeline, history, statistics and a local AI assistant (`app/`, `ai/`).
9. Export a PDF case report and CSV, JSON and PNG files (`core/report_generator.py`).

1.4 Scope of the Project

The scope below lists only what the source code actually does.

In scope (implemented):

- Bitcoin mainnet addresses of the types P2PKH (starting with 1), P2SH (starting with 3), and SegWit or Taproot (starting with bc1), checked with their checksums before any analysis starts.
- Tracing from one to three hops from the root address. Extra hops fetch only the busiest counterparties, so the amount of data stays manageable.
- Five pattern detectors, two clustering rules, four threat or entity categories, and ten risk signals.
- A local SQLite database, seven Streamlit pages, an exportable PDF report and a local AI assistant that uses Ollama.

Out of scope (not implemented):

- Other blockchains, Lightning payments and testnet addresses. BitTrace produces leads, not proof of identity.
- Real-time monitoring, user accounts or login. The tool is a single-user local application.
- A comprehensive sanctions or threat database. Only a small set of verified sample addresses is bundled with the project.

CHAPTER 2 BACKGROUND AND TECHNOLOGIES USED

2.1 How Bitcoin Transactions Work

Bitcoin uses the unspent transaction output (UTXO) model. A transaction takes one or more **inputs**, which are earlier outputs that have not been spent, and creates one or more new **outputs**. Each output pays an amount to an address. An input must be spent completely, so when a sender wants to pay less than the input value, the transaction includes an extra output that sends the remainder back to the sender. This output is called **change**. Understanding change is important for investigators because the change address usually belongs to the sender.

Addresses come in several formats. Addresses beginning with 1 are legacy pay-to-public-key-hash (P2PKH), those beginning with 3 are pay-to-script-hash (P2SH), and those beginning with bc1 are SegWit (Bech32) or Taproot (Bech32m). All of them contain a checksum, which allows software to detect typing mistakes. BitTrace uses this to reject invalid input early.

2.2 Forensic Concepts Used in BitTrace

Table 1: Forensic concepts and the modules that use them

Concept	Simple explanation	Where it is used
Common-input ownership	If several addresses are spent together as inputs of one transaction, they are probably controlled by the same wallet.	clustering.py
Change detection	An output that looks like leftover money (pays back to an input address, same address type as the inputs, not a round amount, or the only new output) probably returns to the sender.	clustering.py
Peel chain	Funds move through a long chain of addresses. At each step a small amount is peeled off to a payee and the rest continues.	pattern_detector.py
Fan-out / fan-in	One address pays many addresses (fan-out), or many addresses pay one (fan-in). Often seen in distribution and collection.	pattern_detector.py
CoinJoin-style	A transaction with several outputs of equal value, which hides which output belongs to which input.	pattern_detector.py
Circular flow	Funds leave an address and come back to it through other addresses, which can hide the real source.	pattern_detector.py
Centrality	Graph measures that show which addresses are important hubs or bridges (PageRank, betweenness).	graph_analytics.py
Threat-list matching	Comparing addresses with published lists of sanctioned, ransomware or darknet addresses.	threat_intel.py

None of these heuristics gives certainty. They produce leads that an investigator must confirm. This limitation is discussed again in Chapter 9.

2.3 Tools and Technologies Used

The project is written entirely in Python. The libraries below are listed in `requirements.txt` or are part of the Python standard library. Python 3.12 was the interpreter in the environment used to run the checks described in Chapter 8.

Table 2: Tools and technologies used in BitTrace

Tool / Technology	Purpose in BitTrace
Python 3	Programming language for all modules
Streamlit	Multi-page web dashboard that runs locally in the browser
requests	HTTPS calls to the Blockstream and mempool.space (Esplora) APIs and to the local Ollama server
NetworkX	Directed multigraph, PageRank, betweenness centrality, cycle search
python-louvain	Louvain community detection (used only if installed)
pandas / NumPy	Tables for the History, Timeline and Statistics pages
Plotly	Interactive graph, timeline and statistics charts in the dashboard
Matplotlib	Static charts and the PNG graph inside exported reports
ReportLab	PDF case report generation
SQLite (sqlite3)	Local database file data/bittrace.db
Ollama	Local large language model server for the AI Assistant (default model qwen3:8b)
sentence-transformers	Optional semantic search for the AI Assistant; keyword search is used if it is absent
hashlib, json, csv, threading	Checksum validation, data export and standard-library support

CHAPTER 3 SYSTEM REQUIREMENTS

3.1 Functional Requirements

The functional requirements were derived from the problems in Section 1.2 and match the behaviour found in the source code.

Table 3: Functional requirements

ID	Requirement	Implemented in
FR1	Accept a Bitcoin address, a hop depth (1 to 3) and optional notes; reject invalid addresses before any network call.	1_Investigate.py, validation.py
FR2	Fetch transactions from a public API with retry and fallback, or use built-in demo data.	data_fetcher.py, demo_data.py
FR3	Cluster related addresses.	clustering.py
FR4	Build the fund-flow graph and compute centrality.	graph_builder.py, graph_analytics.py
FR5	Detect peel chains, fan-out, fan-in, CoinJoin-style and circular flows.	pattern_detector.py
FR6	Match addresses with threat lists and infer entity types.	threat_intel.py, entity_attribution.py
FR7	Score each address 0 to 100 with an itemised explanation.	risk_engine.py
FR8	Save investigations and let the user browse, re-open and delete them.	database.py, 4_History.py
FR9	Show the graph, the timeline and aggregate statistics.	2_Graph_Explorer.py, 3_Timeline.py, 5_Statistics.py
FR10	Answer questions about one saved investigation in plain English using a local model.	ai/ package, 6_AI_Assistant.py
FR11	Export a PDF report and CSV, JSON and PNG files.	report_generator.py

3.2 Non-Functional Requirements

- **Explainability:** every risk score must carry a list of signals with points and reasons.
- **Robustness:** network failures, missing data and a missing AI server must produce a clear message, not a crash.
- **Privacy:** the AI assistant runs locally and the dashboard sends no usage telemetry (`gatherUsageStats = false` in `.streamlit/config.toml`).
- **Honesty about data:** demo data must be clearly labelled as synthetic, and missing block times must never be invented.
- **Simple setup:** one `pip install` and one command (`streamlit run app/Home.py`).

3.3 Software and Hardware Environment

No special hardware is needed. BitTrace runs on an ordinary laptop with Python 3 and the packages in `requirements.txt`. Internet access is needed only for live investigations, and the AI Assistant also needs Ollama.

CHAPTER 4 SYSTEM ARCHITECTURE AND DESIGN

4.1 Overall Architecture

BitTrace follows a layered design with a clear separation of duties. The Streamlit pages in `app/` only collect input and show results. All analysis lives in the `core/` package. `db/` handles storage, `ai/` handles the assistant, and `config.py` holds every threshold and endpoint so that no module hard-codes its own values.

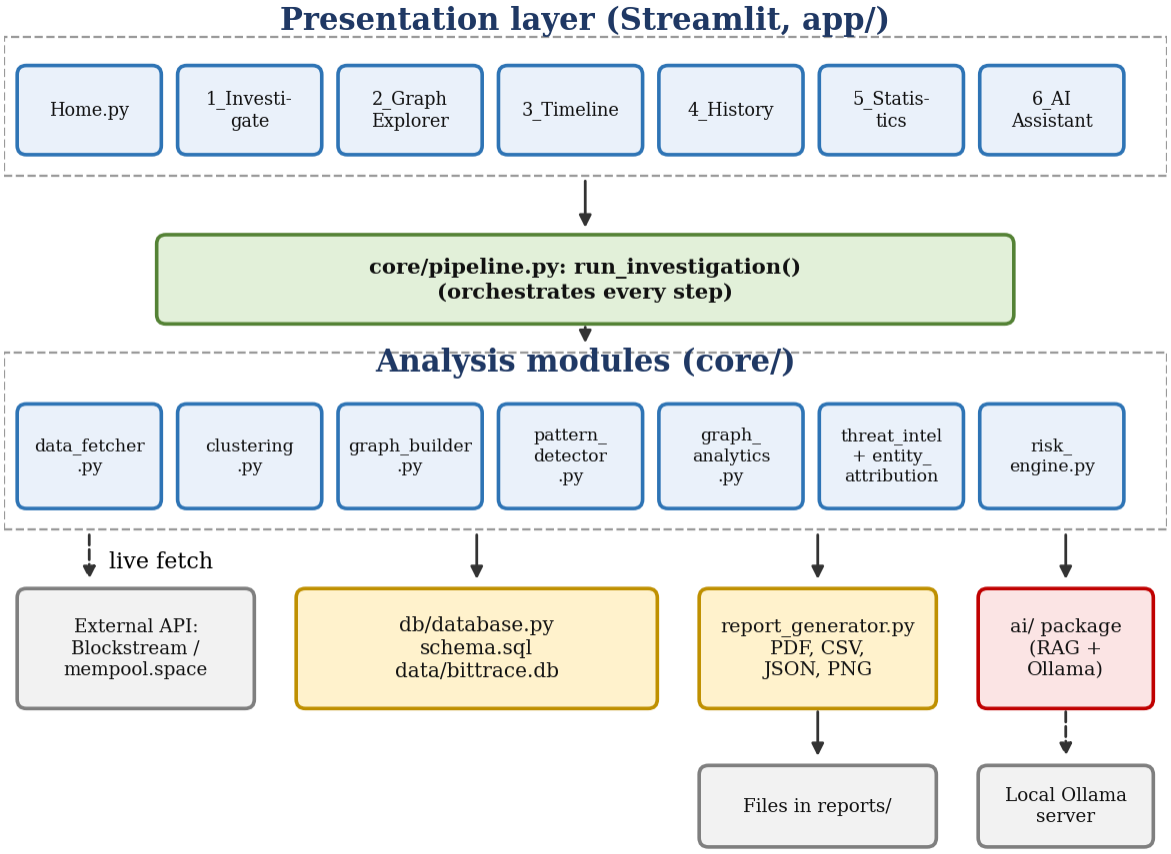


Figure 1: Overall system architecture of BitTrace

Figure 1 shows the main blocks. The page `1_Investigate.py` calls `run_investigation()` in `core/pipeline.py`. This function is the single entry point for the analysis and calls the analysis modules one after another. Results are written to SQLite as they are produced. The other pages later read the saved data back, so an investigation can be re-opened without repeating network calls.

4.2 Module Overview

Table 4: Main source files and their responsibilities

File	Responsibility
config.py	All thresholds, weights, API endpoints, paths and AI settings
core/validation.py	Checksum validation of Bitcoin addresses (Base58Check, Bech32, Bech32m)

File	Responsibility
core/data_fetcher.py	DataFetcher class: API requests, retry, fallback server, pagination, normalize_tx()
core/demo_data.py	build_demo_transactions(): deterministic synthetic transactions for demos and tests
core/clustering.py	UnionFind, detect_change_output(), cluster_transactions(), cluster_summary()
core/graph_builder.py	build_flow_graph(): address-to-address directed multigraph
core/pattern_detector.py	Five detectors and detect_all_patterns()
core/graph_analytics.py	PageRank, betweenness, degree centrality, Louvain communities
core/threat_intel.py	Loads bundled threat lists and builds the lookup index
core/entity_attribution.py	Dataset matches and exchange-like behaviour heuristic
core/risk_engine.py	score_address() and score_all()
core/pipeline.py	run_investigation(): orchestrates every step and saves the results
core/graph_view.py	build_focus_view(): trims the graph for readable display
core/dashboard_data.py	Data preparation for History, Timeline and Statistics pages
core/report_generator.py	PDF, CSV, JSON and PNG export; result_from_database()
db/database.py, db/schema.sql	SQLite connection, schema creation and all queries
ai/ollama_client.py, rag_engine.py, assistant.py, investigation_context.py	Local AI assistant with retrieval-augmented answers
app/Home.py and app/pages/1 to 6	Streamlit user interface
ui_theme.py	Shared dark theme, metric cards, PDF download button

4.3 Data Flow

Figure 2 follows the data rather than the code. Raw API responses are converted into one common transaction format. From this list the system derives clusters and a flow graph. Patterns and graph measures are computed from the graph, and entity attribution combines the graph with the threat lists. The risk engine finally combines patterns, graph measures and attributions into a score for each address.

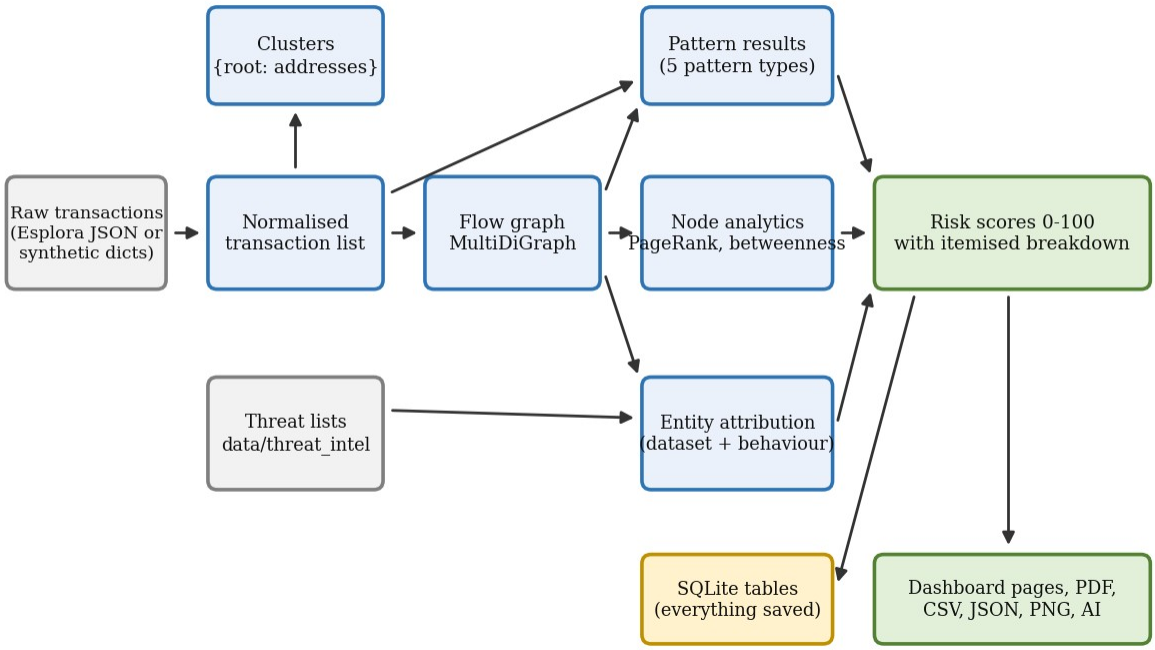


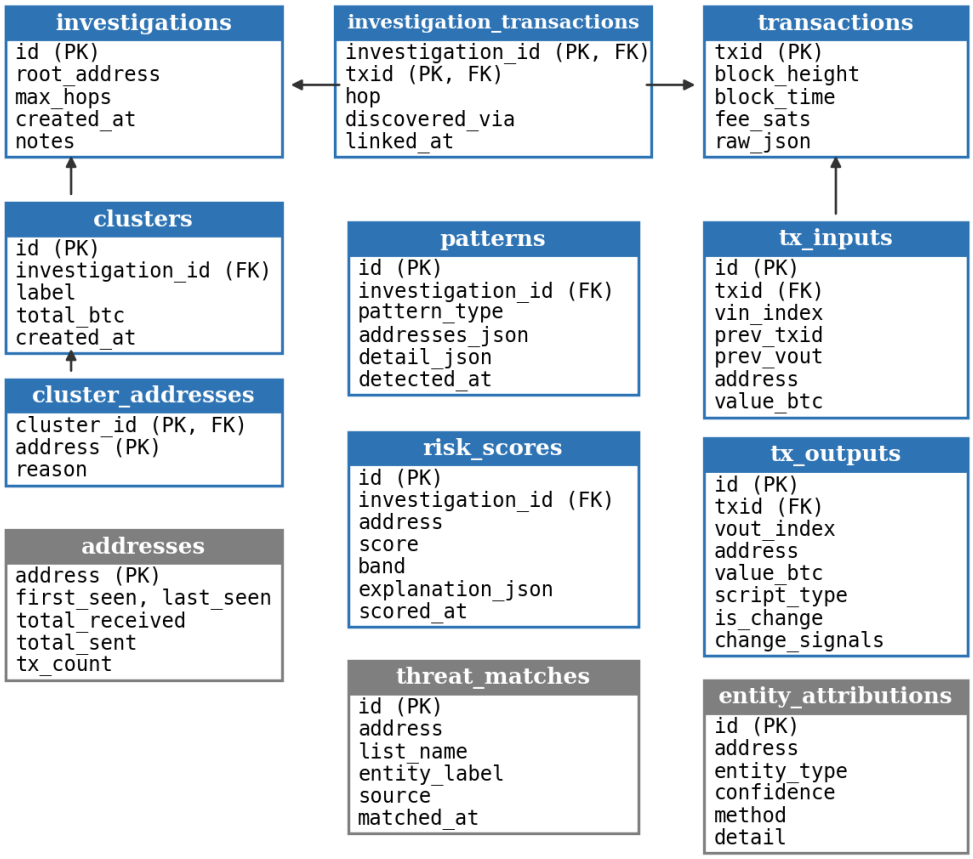
Figure 2: Data-processing flow between the main modules

A normalised transaction has a transaction ID, optional block height and time, a fee, and lists of inputs and outputs. Each input or output carries an address and a value in BTC. Because the demo data uses the same format, every module after the fetcher works identically for live and demo investigations.

4.4 Database Design

BitTrace stores its data in one SQLite file, `data/bittrace.db`. The schema in `db/schema.sql` defines eleven tables. A twelfth table, `investigation_transactions`, is created by the function `_ensure_link_schema()` in `db/database.py`. Figure 3 shows the tables and their relationships.

Grey = address-level tables with no foreign key to an investigation. Blue = all other tables.



Also: patterns and risk_scores reference investigations(id);
tx_outputs references transactions(txid). Child-to-parent arrows shown.

Figure 3: Database structure of BitTrace (SQLite)

Table 5: Database tables and their purpose

Table	Purpose
investigations	One row per run: root address, number of hops, creation time, notes
addresses	Every address seen, with first/last seen and running totals
transactions	One row per transaction ID; raw JSON is kept. Shared between investigations
tx_inputs / tx_outputs	Inputs and outputs of each transaction, with address, value, script type and change flags
investigation_transactions	Link table that records exactly which transactions each investigation analysed
clusters / cluster_addresses	Clusters of each investigation and their member addresses
patterns	Detected patterns with the addresses involved and details as JSON
threat_matches	Addresses found on a bundled threat list
entity_attributions	Entity type, confidence and method for an address
risk_scores	Score, band and the explanation (breakdown) as JSON for each address in an investigation

Two design decisions are worth explaining. First, transactions and addresses are **shared**: if two investigations touch the same transaction it is stored once. The link table keeps each investigation's own set of transactions separate, so graphs and timelines can be rebuilt exactly. Second, investigations that were saved before the link table existed are marked as legacy. For those the History page and the Graph Explorer say clearly that the exact transaction set is unknown instead of guessing.

All database access goes through functions in `db/database.py`. The queries use placeholders (`?`) for values. Where an `IN (...)` list is needed the code builds only the placeholder string, never the data, so user text is never pasted into SQL.

4.5 Configuration Design

Section 2 of `config.py` collects every tunable value. Keeping them in one place makes the detectors easy to explain and adjust. The most important settings are listed below.

Table 6: Key settings in config.py

Setting	Value	Meaning
BLOCKSTREAM_API_BASE	https://blockstream.info/api	Primary Esplora API
API_FALLBACK_BASES	mempool.space/api	Used if the primary server fails
API_TIMEOUT / MAX_RETRIES	15 s / 4	Per request timeout and attempts
MAX_TX_PER_ADDRESS	100	Most transactions fetched for the root address (25 per page)
HOP_MAX_NEIGHBORS	6	Busiest counterparties expanded per extra hop
HOP_MAX_TX_PER_ADDRESS	25	Transactions fetched for each expanded neighbour
PEEL_CHAIN_MIN_HOPS	3	Minimum length of a reported peel chain
FAN_OUT_MIN_OUTPUTS / FAN_IN_MIN_INPUTS	5 / 5	Distinct counterparties needed for a hub
COINJOIN_MIN_EQUAL_OUTPUTS	3	Equal-valued outputs needed in one transaction
RISK_BAND_HIGH / MEDIUM	60 / 35	Score thresholds for the risk bands
RISK_DATASET_MATCH_FLOOR	70	Minimum score for a confirmed threat-list match
OLLAMA_MODEL	qwen3:8b	Default model (can be changed with the OLLAMA_MODEL environment variable)

CHAPTER 5 METHODOLOGY AND INVESTIGATION WORKFLOW

5.1 Overall Workflow

An investigation follows the twelve steps shown in Figure 4. The order was checked against the body of `run_investigation()` in `core/pipeline.py`. Input validation takes place on the Investigate page before the pipeline is called, and only for live investigations, because the demo mode uses a placeholder root address that is not a real Bitcoin address.

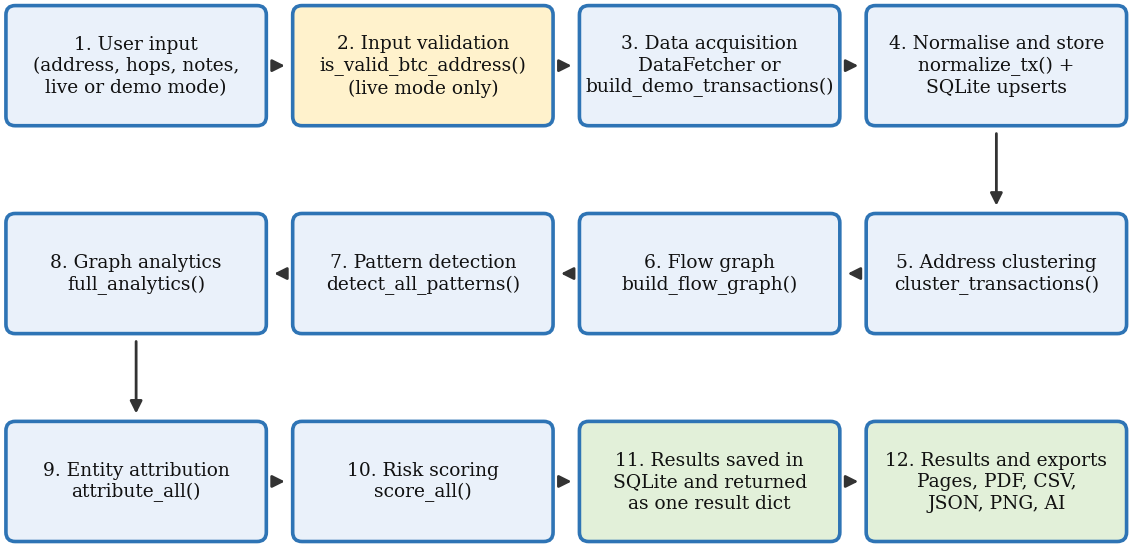


Figure 4: Investigation workflow of BitTrace

5.2 Step-by-Step Explanation

1. **User input.** The analyst enters a root address, chooses 1 to 3 hops (default 2) and may add notes. A tick box selects the built-in demo data instead of a live fetch.
2. **Input validation.** `is_valid_btc_address()` checks length, characters and the checksum. An invalid address stops the process with a message. The text box is also limited to 100 characters.
3. **Data acquisition.** In live mode `DataFetcher` fetches the address summary and up to 100 transactions, and for 2 or 3 hops also the recent transactions of the busiest neighbours. In demo mode `build_demo_transactions()` supplies the data. If the address summary cannot be fetched, the pipeline raises `FetchError` instead of reporting an empty success.
4. **Normalise and store.** Each raw transaction is converted by `normalize_tx()` into the common format. The pipeline creates the investigation row, then saves transactions, inputs, outputs and addresses and links the transactions to the investigation.
5. **Address clustering.** `cluster_transactions()` applies the two clustering rules. Every cluster and its members are saved.

6. **Flow graph.** `build_flow_graph()` creates the address-to-address graph. Cluster totals are then calculated from its edges.
7. **Pattern detection.** `detect_all_patterns()` finds the five pattern types. Each finding is saved and every address involved is remembered for scoring.
8. **Graph analytics.** `full_analytics()` calculates PageRank, betweenness, degree centrality and, if available, communities.
9. **Entity attribution.** Addresses are matched with the threat lists, and exchange-like behaviour is looked for. Matches and attributions are saved.
10. **Risk scoring.** `score_all()` produces the score, band and breakdown for each address, and they are saved.
11. **Result.** The pipeline returns one dictionary with everything. The Investigate page keeps it in the session state for display.
12. **Presentation and export.** The result drives the dashboard, the PDF, CSV, JSON and PNG exports and, through the database, the other pages and the AI assistant.

5.3 Live Mode and Demo Mode

Live mode uses real blockchain data and needs internet access. Demo mode uses `build_demo_transactions()`, which returns a fixed set of 18 invented transactions. The set was written so that every detector has something to find: a root address with a change output, a peel chain, a fan-out hub, a fan-in hub, a CoinJoin-style transaction, a circular flow and an exchange-like hub. It also includes one transaction in which an invented address pays a real WannaCry address from the bundled list. Demo results are marked `synthetic` in the result metadata, and both the Investigate page and the PDF report say that they are not real blockchain data.

Demo mode has two purposes. It lets the tool be shown or tested without internet access, and it gives results that are repeatable, because the same input always produces the same output. It was used for the checks reported in Chapter 8.

5.4 Hop Expansion

With one hop only the root address's own transactions are analysed. For two or three hops, the helper `_expand_hops()` in `core/pipeline.py` looks at the counterparties found so far, selects the busiest ones (at most 6 per hop, set by `HOP_MAX_NEIGHBORS`), and fetches one page of their transactions (at most 25, set by `HOP_MAX_TX_PER_ADDRESS`). The number of expanded addresses is stored in the result metadata and shown to the user. This limit keeps API use and waiting time under control, but it also means that a deeper trace is a sample of the neighbourhood and not a complete crawl.

CHAPTER 6 SYSTEM IMPLEMENTATION

This chapter explains the main modules. For each one it describes the input, the processing and the output, and quotes short parts of the real code where this helps understanding.

6.1 Input Validation (core/validation.py)

Input: the text typed by the user. **Output:** `True` or `False`. The function `is_valid_btc_address()` first removes surrounding spaces and rejects text that is too short, too long, not ASCII or containing spaces. Addresses starting with 1 or 3 are passed to `_base58check_ok()`, which decodes the Base58 string and compares a double SHA-256 checksum. Everything else goes to `_bech32_ok()`, which accepts only the `bc1` prefix and verifies the Bech32 or Bech32m checksum.

```
def is_valid_btc_address(addr: str) -> bool:
    if not isinstance(addr, str):
        return False
    addr = addr.strip()
    if not 26 <= len(addr) <= 74 or not addr.isascii() or " " in addr:
        return False
    if addr[0] in "13":
        return _base58check_ok(addr)
    return _bech32_ok(addr)
```

Because the checksum is verified, a typing mistake is caught locally, and strings such as SQL fragments never reach the network layer or the database. Testnet addresses (`tb1...`) are rejected because the tool is built for mainnet.

6.2 Blockchain Data Retrieval (core/data_fetcher.py)

The `DataFetcher` class talks to the Esplora API offered by Blockstream and, as a fallback, by mempool.space. Its central method `_get(path)` implements the retry logic shown in Figure 5. A 404 or 400 response is not retried on the same server, a 429 (rate limit) response waits for the time given by the server or a backoff delay, and other errors, timeouts and SSL errors are retried with a growing delay. If every server and attempt fails, the method returns `None` and the reason for each failing server is kept in `last_error`.

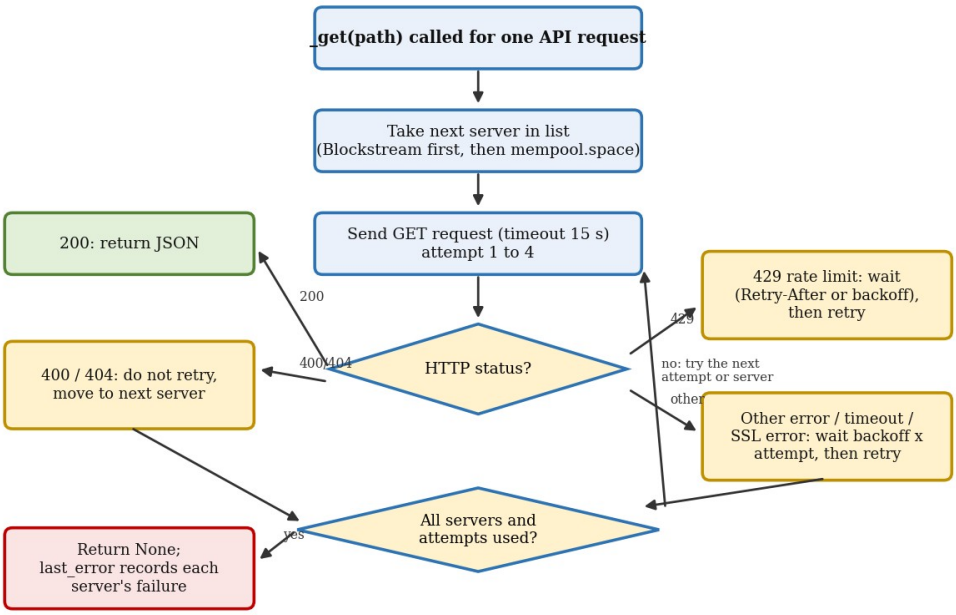


Figure 5: Request, retry and fallback logic of DataFetcher._get()

`get_address_txs()` follows the Esplora pagination: the first call returns recent transactions including unconfirmed ones, and the following calls use the last confirmed transaction ID as a cursor, 25 transactions at a time, until the limit of 100 is reached or a short page ends the list. `normalize_tx()` converts each raw transaction into the internal format, including value conversion from satoshis to BTC. When SSL errors occur, the final error message includes a hint that antivirus web shields, VPNs or restricted networks are a common cause.

6.3 Address Clustering (core/clustering.py)

Input: the list of normalised transactions. **Output:** a dictionary of clusters, later turned into a sorted list by `cluster_summary()`. A small union-find structure merges addresses. Two rules are applied to every transaction.

```
for tx in txs:
    input_addrs = [i["address"] for i in tx["inputs"] if i.get("address")]
    # Rule 1: common-input-ownership
    if len(input_addrs) > 1:
        for other in input_addrs[1:]:
            uf.union(input_addrs[0], other)
    # Rule 2: change-address detection
    for o in detect_change_output(tx, set(input_addrs)):
        if o["is_change"] and o.get("address") and input_addrs:
            uf.union(input_addrs[0], o["address"])
```

`detect_change_output()` gives an output one point for each of four signals: it pays back to one of the input addresses, it has the same script type as the inputs, its amount is not a round number, and it is the only output that is not also an input address. An output is treated as change when it reaches at least two points (`CHANGE_HEURISTIC_MIN_SIGNALS`). Round amounts are multiples of 1, 0.1, 0.01 or 0.001 BTC. The result of this check is stored with the output in the `is_change` and `change_signals` columns.

In the cluster list, pure recipients that never appear as inputs and are not detected as change do not form clusters of their own. This is why the demo run reports 29 cluster members for 45 addresses.

6.4 Flow Graph (core/graph_builder.py)

`build_flow_graph()` returns a NetworkX `MultiDiGraph` in which every node is an address. For each transaction, an edge is drawn from each input address to each output address. Each edge carries a share of the output value equal to that input's share of the total input value (`share = input_value / total_input x output_value`), and it stores `value_btc`, `txid`, `block_time` and an `is_change` flag. Edges from an address to itself are skipped. Because the split is proportional, edge values are estimates of flow and not exact coin tracing. This is a known approximation of the method.

6.5 Pattern Detection (core/pattern_detector.py)

`detect_all_patterns(g, txs)` returns a dictionary with five lists. Table 8 summarises the rules and the values taken from `config.py`.

Table 7: Pattern detection rules

Pattern	Rule used in the code	Threshold
Peel chain	Starting from an address with exactly two outgoing edges, follow the larger one while each next address also has exactly two outgoing edges. Stops on a revisited address (cycle guard).	At least 3 hops; cap 200
Fan-out	Address with many distinct recipients	5 or more targets
Fan-in	Address with many distinct senders	5 or more sources
CoinJoin-style	A transaction in which several outputs have equal value (within 1e-8 BTC)	3 or more equal outputs
Circular flow	Simple cycles in the flow graph found with NetworkX	Length 2 to 10

The CoinJoin-style rule looks only at equal output values. It does not check how many different inputs the transaction has, so it is a loose signal. The testing chapter shows one effect of this.

6.6 Graph Analytics (core/graph_analytics.py)

`full_analytics(g)` calculates, for every node, the PageRank (weighted by `value_btc`), the normalised betweenness centrality and the degree centrality. If the optional package `python-louvain` is installed it also assigns a community number. If it is missing the community result is empty and the rest of the program continues. Betweenness is used by the risk engine: an address with betweenness above 0.1 is treated as a pass-through hub.

6.7 Threat Intelligence and Entity Attribution

`core/threat_intel.py` loads three JSON files from `data/threat_intel/`: `OFAC_SDN.json`, `WannaCry.json` and `DarknetMarket.json`. Together they contain eight addresses. Each entry in the source code is documented with a citation, and the file `data/threat_intel_README.txt` explains how to extend the lists. The home page of the application states plainly that this is a small sample and not a complete sanctions database.

Table 8: Bundled threat lists

List	Entity type	Addresses
OFAC_SDN.json	sanctioned	4
WannaCry.json	ransomware	3
DarknetMarket.json	darknet_market	1

`core/entity_attribution.py` produces attributions in two ways. A **dataset match** is created when the address is in one of the lists, with a confidence of 0.95. A **behavioural heuristic** labels an address as an exchange-like hub when it has outgoing transactions and at least five distinct counterparties sending to it. The confidence of this label is `min(0.9, 0.4 + 0.05 x senders)` with senders capped at 10, so it can never exceed 0.9. The label is a guess from behaviour and is stored as such, with the method name `behavioral_heuristic`.

6.8 Risk Scoring (core/risk_engine.py)

Input: for one address, its entity attributions, the patterns it takes part in and its betweenness. **Output:** a dictionary with `score`, `band` and a `breakdown` list. Every signal adds or subtracts points and writes its own line in the breakdown, so a reader can recompute the final score by hand.

Table 9: Risk signals and their weights

Signal	Points	Condition
sanctioned_match	+45	Address is on a sanctions list
ransomware_match	+35	Address is on a ransomware list
darknet_match	+25	Address is on a darknet market list
peel_chain	+10	Address is in a detected peel chain
circular_flow	+10	Address is in a detected cycle
coinjoin	+8	Address is in a CoinJoin-style transaction
fan_out / fan_in	+6 each	Address is a fan-out or fan-in hub or one of its counterparties
high_betweenness	+8	Betweenness greater than 0.1
exchange_heuristic	-5	Exchange-like behaviour and not on a threat list
confirmed_threat_floor	to 70	On a threat list: score is raised to at least 70

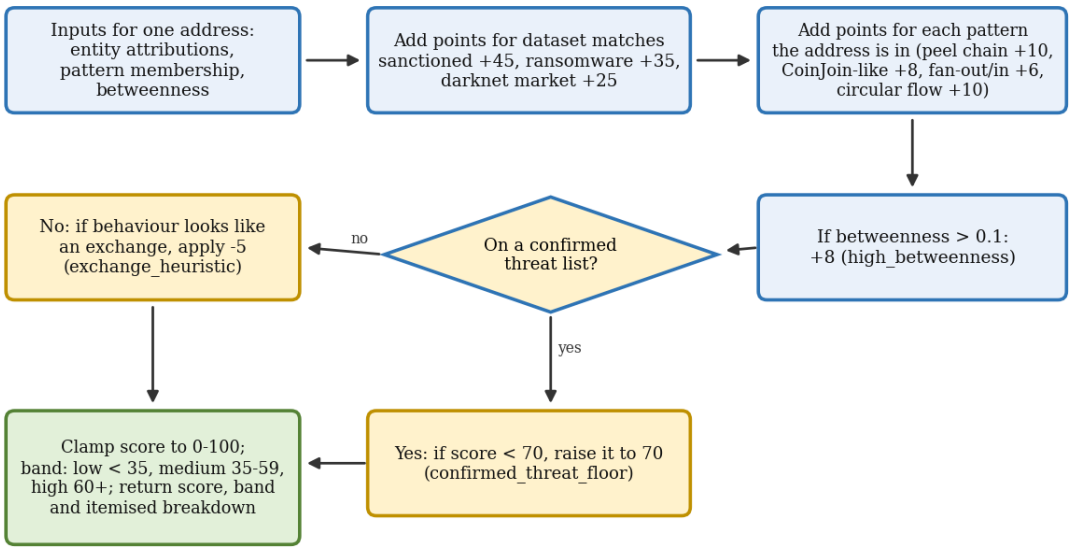


Figure 6: Decision flow of the risk engine

```
if confirmed_threat and score < config.RISK_DATASET_MATCH_FLOOR:
    lift = config.RISK_DATASET_MATCH_FLOOR - score
    score += lift
    breakdown.append({"signal": "confirmed_threat_floor", "points": lift, ...})
score = max(0, min(config.RISK_SCORE_MAX, score))
```

The code shows two design choices. The floor is written to the breakdown as its own line, so the displayed points still add up to the score. The exchange discount is never applied to an address on a threat list, so looking like an exchange cannot hide a documented match. The score is then clamped to 0 to 100. The risk bands are low (below 35), medium (35 to 59) and high (60 and above).

6.9 Data Storage (db/database.py)

`init_db()` runs `schema.sql`, enables foreign keys and creates the link table if it is missing. Connections come from `get_conn()`, a context manager, so each call opens and closes its own connection. Functions such as `upsert_transaction()`, `create_cluster()`, `save_pattern()`, `save_risk_score()` and `link_transactions()` write the data, and `get_investigation_summaries()` runs one aggregated query for the History and Statistics pages. `delete_investigation()` removes an investigation's clusters, patterns, risk scores and links but keeps the shared transaction data.

6.10 PDF Report Generator (core/report_generator.py)

`build_pdf_report(result)` creates the PDF in memory with ReportLab. `export_all(result)` writes the PDF, a CSV of risk scores, a JSON copy of the result and a PNG of the graph into `reports/`. `result_from_database(id)` rebuilds a result from stored records so an old investigation can still be reported.

Table 10: Sections of the PDF case report

No.	Section	Content
-	Title block	Investigation ID, dates, root address, hops, data source, notes
1	Summary of findings	Counts of transactions, addresses, clusters, patterns and risk bands, plus key findings
2	Risk assessment	Bar chart by band, table of high-risk addresses, highest-scoring addresses
3	Graph analysis and patterns	Focus view of the graph, most central addresses, pattern list
4	Address clusters	Largest clusters
5	Transaction details	Transactions of the analysis (when available)
6	Method and limitations	How the results were produced and what they cannot show

Each section is generated inside its own protection block, so one failing section does not stop the report. If block times are unknown, the report says so. A report built from stored records states that transactions and the graph are not included, because they are not saved per investigation.

6.11 Local AI Assistant (ai/ package)

The assistant answers questions about one saved investigation. It does this with retrieval-augmented generation (RAG). `investigation_context.py` loads the saved risk scores, patterns, clusters, threat matches and entity attributions of **one** investigation from SQLite and turns them into short text facts. For each question `rag_engine.py` selects the most relevant facts (8 by default, `RAG_TOP_K`) plus a fixed overview, ranking by meaning if sentence-transformers is installed and by keyword overlap otherwise. `assistant.py` builds a prompt from rules, the facts and the last few messages, and `ollama_client.py` sends it to the local Ollama `/api/chat` endpoint. `strip_reasoning()` removes any reasoning text before the answer is shown.

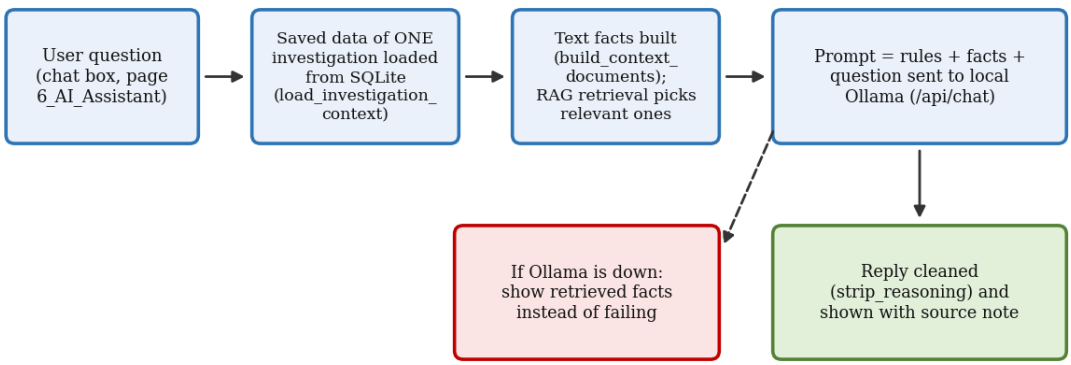


Figure 7: Question-answering flow of the AI assistant

Facts come only from the selected investigation, so two cases cannot be mixed. If Ollama or the model is missing, the assistant returns a clear message with the retrieved facts.

CHAPTER 7 APPLICATION INTERFACE AND FEATURES

The dashboard is started from the project folder with `streamlit run app/Home.py` or with `run_bittrace.bat`. It uses a dark theme defined in `.streamlit/config.toml` and `ui_theme.py`. The sidebar lists the pages below. This chapter describes each page from its source code. Real dashboard screenshots could not be captured in the environment used to prepare this report, because Streamlit was not installed there. The two figures at the end of the chapter are therefore taken from the PDF report that BitTrace itself generated during the demo run.

Table 11: Pages of the BitTrace dashboard

Page (file)	Main features
Home (Home.py)	Overview of the four objectives and modules, the five most recent investigations, a note on the limits of the threat lists
Investigate (1_Investigate.py)	Address box, hop slider (1 to 3), notes, demo-data option; result overview cards, risk table, pattern bars, cluster table, PDF download and Export button
Graph Explorer (2_Graph_Explorer.py)	Interactive fund-flow graph for any saved investigation; search, hops, addresses-per-side slider, three layouts, risk filter, details panel
Timeline (3_Timeline.py)	Transactions over time with Incoming, Outgoing and Related markers, linear or log value scale, transaction table and inspector
History (4_History.py)	Table of all investigations with status and risk counts, details view, PDF report, delete with confirmation
Statistics (5_Statistics.py)	Totals and charts across all, latest-per-address or one investigation: risk bands, patterns, score distribution
AI Assistant (6_AI_Assistant.py)	Chat about one saved investigation; model status check; suggested questions; clear conversation

7.1 Investigate Page

This is the starting point. The form accepts the root address (limited to 100 characters), the maximum number of hops and notes. After pressing **Run investigation** the page checks the input: it asks for an address or the demo option, and for live runs it shows an error if the checksum is invalid. A spinner is shown while `run_investigation()` runs. A `FetchError` is shown as a readable message and any other exception is shown with details, so the page never fails silently. On success the page shows a case banner, five metric cards (transactions analysed, addresses seen, clusters, patterns found and high-risk addresses), a sorted risk table with a coloured band for every address, a bar summary of pattern counts and the largest clusters. Text taken from data is passed through `html.escape()` before it is placed in HTML. The results are kept in `st.session_state` so they stay on screen after the form is submitted.

7.2 Graph Explorer

The Graph Explorer shows the flow graph of the latest or any saved investigation. A full graph can have hundreds of nodes, so the page uses `build_focus_view()` from `core/graph_view.py`. It places the chosen centre address in the middle, senders on the left and recipients on the right, and keeps only the highest-risk and highest-value addresses on each

side. The user can change the number of hops (1 or 2) and the number of addresses per side (5 to 40), choose a flow, force-directed or radial layout, hide lower-risk addresses, search by full or partial address, click an address to see its risk breakdown, entity attributions, cluster size and patterns, and re-centre the graph on it. An expandable section explains how to read the graph. For investigations that are no longer in the session, the page rebuilds the graph from the database and tells the user whether the links are exact or reconstructed.

7.3 Timeline, History and Statistics

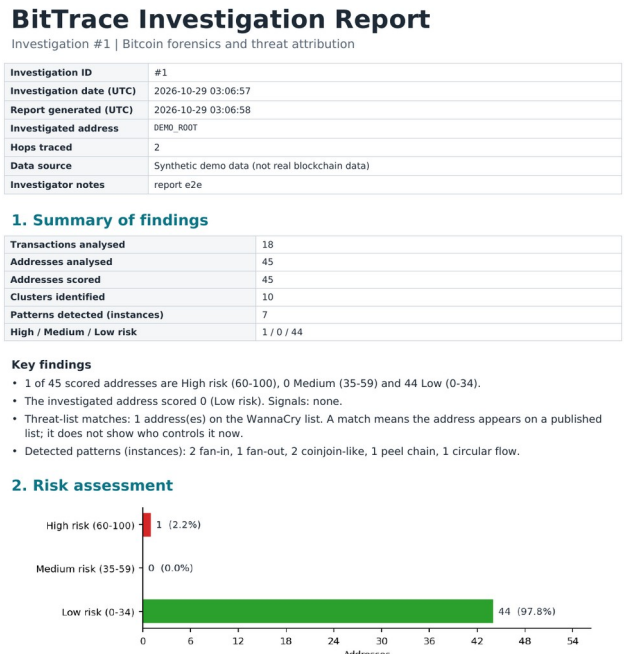
The **Timeline** page uses `timeline_frame()` and `classify_tx()` from `core/dashboard_data.py`. Each transaction is marked Outgoing (the investigated address is a sender), Incoming (it only receives) or Related (it is not in the transaction and was fetched while tracing a neighbour). When block times are not available, the page lists transactions in processing order and does not invent a time. The **History** page shows one row per investigation with its status (completed or incomplete), the number of addresses scored and the high, medium and low counts. For a selected investigation it shows notes, the highest-scoring addresses, the detected patterns and a PDF download button, and it contains a delete option that needs a confirmation tick. The **Statistics** page offers three scopes: all saved investigations, the latest run of each address, or a single investigation. It shows summary cards and charts of the risk bands, pattern counts and the distribution of scores, with an optional logarithmic axis.

7.4 AI Assistant Page

The page asks the user to pick a saved investigation, checks the Ollama server (`ok`, `down` or `no_model`) and shows a matching warning, for example the exact `ollama pull` command if the model is missing. It lists suggested questions, keeps a separate conversation for each investigation and clears it if the saved data changes. Each answer is followed by a caption stating which investigation it is based on.

7.5 Exported PDF Report

Figures 8 and 9 show parts of the PDF that BitTrace generated for the demo investigation. Figure 8 shows the title block, the summary of findings and the risk assessment. Figure 9 shows the focus view of the fund-flow graph and the most central addresses. In this demo run the WannaCry address is the only high-risk address, with a score of 70.



BitTrace Investigation Report

Investigation #1 | Bitcoin forensics and threat attribution

Investigation ID	#1
Investigation date (UTC)	2026-10-29 03:06:57
Report generated (UTC)	2026-10-29 03:06:58
Investigated address	DEMO_ROOT
Hops traced	2
Data source	Synthetic demo data (not real blockchain data)
Investigator notes	report e2e

1. Summary of findings

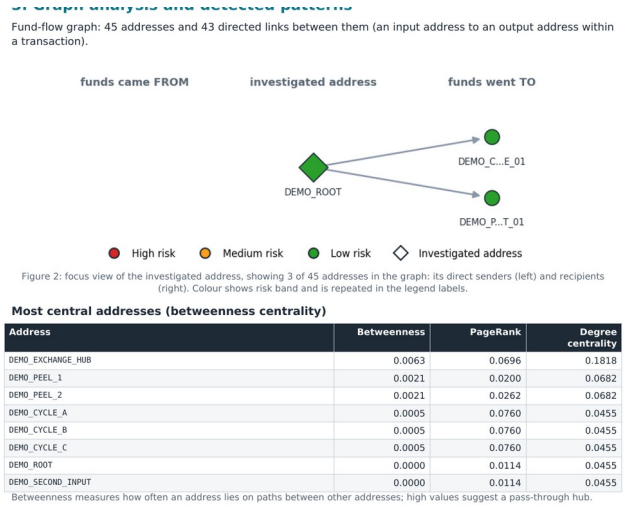
Transactions analysed	18
Addresses analysed	45
Addresses scored	45
Clusters identified	10
Patterns detected (instances)	7
High / Medium / Low risk	1 / 0 / 44

Key findings

- 1 of 45 scored addresses are High risk (60-100), 0 Medium (35-59) and 44 Low (0-34).
- The investigated address scored 0 (Low risk). Signals: none.
- Threat-list matches: 1 address(es) on the WannaCry list. A match means the address appears on a published list; it does not show who controls it now.
- Detected patterns (instances): 2 fan-in, 1 fan-out, 2 coinjoin-like, 1 peel chain, 1 circular flow.

2. Risk assessment

Figure 8: Title block, summary and risk assessment of the PDF report generated for the demo investigation



Fund-flow graph: 45 addresses and 43 directed links between them (an input address to an output address within a transaction).

Figure 2: focus view of the investigated address, showing 3 of 45 addresses in the graph: its direct senders (left) and recipients (right). Colour shows risk band and is repeated in the legend labels.

Most central addresses (betweenness centrality)

Address	Betweenness	PageRank	Degree centrality
DEMO_EXCHANGE_HUB	0.0063	0.0696	0.1818
DEMO_PEEL_1	0.0021	0.0200	0.0682
DEMO_PEEL_2	0.0021	0.0262	0.0682
DEMO_CYCLE_A	0.0005	0.0760	0.0455
DEMO_CYCLE_B	0.0005	0.0760	0.0455
DEMO_CYCLE_C	0.0005	0.0760	0.0455
DEMO_ROOT	0.0000	0.0114	0.0455
DEMO_SECOND_INPUT	0.0000	0.0114	0.0455

Betweenness measures how often an address lies on paths between other addresses; high values suggest a pass-through hub.

Figure 9: Graph focus view and most central addresses in the PDF report

CHAPTER 8 TESTING AND RESULTS

8.1 Testing Approach

Testing for this report had three parts. First, the project's own automated test suite in `tests/` was run. Second, additional end-to-end checks were executed on the demo investigation, input validation, offline behaviour, exports and the AI fallback. Third, features that could not be run in the test environment are listed separately so that they are not mistaken for tested features.

The checks were run in a sandbox without internet access, with Python 3.12 and without pytest, Streamlit, Plotly or python-louvain installed. Because pytest could not be installed, a small stand-in test runner was written that supports the pytest features used by the test files (fixtures, `tmp_path`, `monkeypatch`, `raises`, `approx` and `parametrize`). The test files themselves were run without changes.

8.2 Project Test Suite

Table 12: Automated tests in the tests/ folder (executed)

Test file	What it checks	Passed / run
test_ai_context.py	AI context isolation per investigation, prompt content, offline fallback	6 / 6
test_clustering.py	Common-input ownership, change detection, round amounts, union-find	4 / 4
test_dashboard_data.py	Timeline frame, labels, scope selection, band counts, thresholds	12 / 12
test_database.py	Table creation, round trips, delete keeps shared transactions	6 / 6
test_demo_data.py	Demo data is deterministic and exercises the detectors	2 / 2
test_entity_attribution.py	Dataset match, exchange heuristic conditions	4 / 4
test_fetcher_and_hops.py	Pagination cursor, short page stop, hop expansion	3 / 3
test_graph_view.py	Focus view caps sides, prefers high risk, unknown root	4 / 4
test_pattern_detector.py	Peel chain, fan-out, fan-in, CoinJoin, circular flow	7 / 7
test_pdf_report.py	PDF builds, no invented times, empty data, stored-record report	10 / 10
test_risk_engine.py	Bounds, breakdown, threat floor, exchange discount rule	7 / 7
test_validation_and_resilience.py	Address validation, mirror fallback, rate limit, error messages	8 / 8
Total	12 test files	73 / 73

All 73 test cases passed in this run. The README of the project mentions a smaller number of tests, which means it was written before the suite grew; the number above is the count actually collected and run for this report. The Louvain community step was not exercised, because python-louvain was not installed.

8.3 End-to-End Checks on the Demo Investigation

The demo investigation was run through `run_investigation("DEMO_ROOT", max_hops=2, synthetic_txs=build_demo_transactions("DEMO_ROOT"))` with a temporary database. The run took about 0.3 seconds. Its results are summarised below.

Table 13: Records created by the demo investigation (executed)

Item	Result
Transactions analysed	18
Addresses (graph nodes) / edges	45 / 43
Clusters (sizes)	10 (7, 7, 3, 3, 3, 2, 1, 1, 1, 1)
Patterns found	1 peel chain, 1 fan-out, 2 fan-in, 2 CoinJoin-style, 1 circular flow (7 in total)
Risk bands (high / medium / low)	1 / 0 / 44
Highest score	70 (WannaCry address 13AM4V...): ransomware_match +35, confirmed_threat_floor +35
Entity attributions	WannaCry address: ransomware (dataset match, 0.95); DEMO_EXCHANGE_HUB: exchange (behavioural heuristic, 0.7)
Rows saved	investigations 1, addresses 45, transactions 18, tx_inputs 26, tx_outputs 30, clusters 10, cluster_addresses 29, patterns 7, threat_matches 1, entity_attributions 2, risk_scores 45, investigation_transactions 18
Exports	JSON 15,693 bytes; CSV 1,368 bytes; PNG 304,900 bytes; PDF 143,955 bytes (4 pages)

The table of executed tests below uses these results. The status column shows PASS only when the actual result matched the expected result written in advance. One case is marked as an observation because the result was different from what a reader might expect.

Table 14: Executed test cases

ID	Module	Test scenario	Expected result	Actual result	Status
E01	validation	Valid legacy address 1A1zP1eP5Q...DivfNa	Accepted	True	PASS
E02	validation	Valid P2SH address 3J98t1WpEZ...WNLy	Accepted	True	PASS
E03	validation	Valid Bech32 address bc1qar0srr...mdq	Accepted	True	PASS
E04	validation	Valid Taproot address bc1p5d7rjq...3297	Accepted	True	PASS
E05	validation	Valid address with last character changed; one character swapped in the middle	Rejected (checksum)	False in both cases	PASS
E06	validation	Empty text, random text, 80 x characters, and an address followed by '; DROP TABLE'	Rejected	False in all four cases	PASS

ID	Module	Test scenario	Expected result	Actual result	Status
E07	validation	Testnet address tb1qw508d6...	Rejected (mainnet only)	False	PASS
E08	pipeline	Demo investigation, 2 hops	Completes and saves results	18 transactions, 45 addresses, 10 clusters, 7 pattern instances, 45 scores in 0.32 s	PASS
E09	risk engine / threat intel	WannaCry address inside demo data	High band, score at least 70	Score 70, band high, signals ransomware_match and confirmed_threat_floor	PASS
E10	clustering	Root, second input and change output in one transaction	One cluster with the three addresses	Cluster of 3: DEMO_ROOT, DEMO_SECOND_INPUT, DEMO_CHANGE_01	PASS
E11	pattern detector	Planted peel chain of 4 addresses	1 peel chain, 3 hops	1 chain, 3 hops, DEMO_PEEL_0 to DEMO_PEEL_3	PASS
E12	pattern detector	Planted 3-address cycle	1 circular flow	DEMO_CYCLE_A, B, C reported	PASS
E13	pattern detector	Planted fan-out hub (6 targets) and fan-in hub (6 sources)	Fan-out for the fan-out hub; fan-in for the fan-in hub and the exchange hub (6 sources each)	Fan-out: DEMO_FANOUT_HUB (6). Fan-in: DEMO_FANIN_HUB (6) and DEMO_EXCHANGE_HUB (6)	PASS
E14	pattern detector	Planted CoinJoin-style transaction (3 equal outputs)	demo_coinjoin_001 flagged; no other transaction flagged	demo_coinjoin_001 flagged, and demo_fanout_001 (6 equal outputs) also flagged	OBSERVED
E15	entity attribution	Exchange-like hub with 6 senders and outgoing payments	exchange attribution, confidence at most 0.9	exchange, behavioural_heuristic, 0.7	PASS
E16	data fetcher / pipeline	Valid address in live mode with no internet	Clear FetchError, no crash, no empty success	FetchError naming blockstream.info and mempool.space (ConnectionError)	PASS
E17	report generator	export_all() on the demo result	PDF, CSV, JSON and PNG created	All four files created; PDF has 4 pages	PASS
E18	report generator	Data-source line in PDF page 1	States that data is synthetic	'Synthetic demo data (not real blockchain data)'	PASS
E19	AI assistant	Question asked while Ollama is not running	Clear message plus saved facts, no crash	'[AI assistant unavailable] Could not reach Ollama...' followed by facts from the investigation	PASS
E20	tests/ folder	Run all test files	All pass	73 of 73 passed	PASS

8.4 Discussion of Results

The checks show that the modules work together on the demo data and that the planted structures were found. The one result that needs an explanation is E14. The CoinJoin-style detector flags any transaction with three or more equal outputs, so the fan-out transaction of the

demo, which pays six equal amounts, is also flagged. In real data this means that ordinary batch payments with equal amounts can be reported as CoinJoin-style. The report and the risk table call the pattern CoinJoin-style, not CoinJoin, and it adds only 8 points, which limits the effect, but a stricter rule that also looks at the number of distinct inputs would be a sensible improvement.

The risk scores in the demo are low for everything except the threat-list match: pattern signals add 6 to 10 points each, so pattern findings alone never reach the medium band of 35. This follows directly from the weights in `config.py`. It means that in the current design only a combination of several signals or a dataset match can raise an address to medium or high. Whether these weights are well calibrated for real cases was not measured, because no labelled real-world data set was used.

8.5 Features Not Tested in This Environment

The following parts of the system exist in the source code but were not executed for this report. No result is claimed for them.

Table 15: Items not executed

Item	Reason	Status
Live fetching from Blockstream / mempool.space	No internet access in the test environment; only the failure path (E16) and the mocked retry logic in the test suite were run	NOT EXECUTED
Streamlit pages and all charts on screen	Streamlit and Plotly were not installed; no screenshots were taken	NOT EXECUTED
AI answers from a language model	Ollama and the model were not available; only the fallback path (E19) was run	NOT EXECUTED
Semantic retrieval with sentence-transformers	Library not installed	NOT EXECUTED
Louvain community detection	python-louvain not installed	NOT EXECUTED
Detection accuracy on real laundering cases	No labelled real-world data set was used; no accuracy figures are claimed	NOT EXECUTED
Performance with large addresses (thousands of transactions)	Not measured	NOT EXECUTED

CHAPTER 9 SECURITY CONSIDERATIONS AND LIMITATIONS

9.1 Security Measures in the Implementation

- **Input validation:** addresses are checksum-verified before any API call or database write, and the address box has a length limit.
- **SQL safety:** queries use bound parameters. The only dynamically built SQL pieces are lists of `?` placeholders and fixed table names from the code.
- **Output encoding:** data inserted into custom HTML on the Investigate page passes through `html.escape()`.
- **Secure transport:** all API calls use HTTPS, and the user is told when SSL fails.
- **No secrets:** the tool needs no API keys and has none stored.
- **Local AI:** the assistant talks only to the local Ollama server, and the facts it receives come from one investigation.
- **No telemetry:** Streamlit usage statistics are switched off.
- **Safe failure:** the pipeline raises an error instead of reporting an empty investigation as a success, and the PDF generator isolates sections so one failure does not lose the whole report.
- **Evidence honesty:** demo data is labelled as synthetic, missing block times are shown as unknown, and the threat-list match is described as a match with a published list and not as proof of control.

9.2 Privacy and Operational Considerations

- In live mode the investigated addresses are sent to Blockstream or mempool.space. Those services can therefore see which addresses were looked up. This matters for sensitive investigations.
- The database `data/bittrace.db` and the files in `reports/` are not encrypted. They should be protected with normal operating-system access control.
- The LAN launcher starts Streamlit on `0.0.0.0`, which makes the dashboard reachable from other computers on the network. The application has no login, so it should only be used on a trusted network.
- The reports are not digitally signed. A reader cannot prove from the file alone that it has not been changed.

9.3 Limitations

- **Heuristics are not proof.** Common-input ownership and change detection can be wrong, for example when users combine coins in a CoinJoin or when the change output is made to look like a payment. Wrong merges can join unrelated owners.
- **Sampling.** The root address is limited to its most recent 100 transactions, and extra hops fetch only the busiest 6 neighbours and 25 transactions each. Very active addresses are therefore analysed only in part. The page states how many transactions exist and how many were used.
- **Approximate flow values.** Edge values come from a proportional split of each transaction and are not exact coin tracing.

- **Loose CoinJoin rule.** As shown in test E14, any transaction with three or more equal outputs can be flagged.
- **Small threat lists.** Only eight addresses are bundled, so most real threats will not be matched. An optional Ransomwhere feed exists in the code but is marked unverified in `config.py` and is not used by the pipeline by default.
- **Unvalidated weights.** The score weights are expert choices and were not calibrated on labelled data. The tool gives no accuracy or false-positive rate.
- **Storage gaps.** The graph and its analytics are not stored, so they cannot be shown for investigations saved before the link table existed, and the Graph Explorer then explains the difference.
- **Scope.** Bitcoin mainnet only, one user at a time, no live monitoring.
- **AI assistant.** A language model can still misread facts. The answer is an aid for reading the stored data and must be checked against the tables and the report.
- **Documentation mismatch.** The README mentions a different default model name than `config.py` (which uses `qwen3:8b`). The configuration file is the one the program uses.

CHAPTER 10 CONCLUSION AND FUTURE IMPROVEMENTS

10.1 Conclusion

BitTrace brings the main steps of a Bitcoin investigation into one local tool. From a single address it downloads the surrounding transactions, groups addresses that probably belong together, builds the flow graph, looks for peel chains, fan-out and fan-in hubs, CoinJoin-style transactions and circular flows, checks addresses against threat lists and gives every address a risk score that can be explained line by line. Each investigation is stored in SQLite, can be explored in the dashboard and is exported as a PDF case report.

The project reached all of its stated objectives. On the demo investigation every planted structure was found, the threat-list address was scored 70 and placed in the high band, the results were saved to twelve tables and four export files were produced. The 73 test cases of the project passed, and the failure paths for no internet and no AI server were shown to behave in a controlled way. The main strengths are the explainable scoring, the careful handling of missing data and the clear separation between demo and live data.

The testing also showed the limits of the work. The detectors are heuristics, the weights were not calibrated on real labelled cases, the bundled threat lists are small and the live network and dashboard screens were not exercised in the test environment. These points should be stated clearly in the viva, and they lead naturally to the improvements below.

10.2 Future Improvements

The following items are **not implemented** and are proposals only.

- Make the CoinJoin rule stricter by also requiring several distinct inputs and a minimum number of equal outputs relative to the total.
- Calibrate the risk weights on a labelled set of known cases and report precision and recall.
- Import a larger sanctions and threat-intelligence feed, for example by completing the optional OFAC SDN download described in the README.
- Store the graph analytics per investigation so that old cases can show communities and centrality.
- Add support for other chains and for Lightning-related heuristics.
- Add login and encryption for the database for shared or multi-user use, and digital signing or hashing of exported reports.
- Add a regression test for the full Streamlit pages using an automated browser, and capture screenshots for documentation.
- Allow the user to choose the AI model and to export the AI conversation together with the case report.

In summary, BitTrace is a working and explainable first version of a Bitcoin forensics assistant, and the structure of its modules and configuration makes each of these improvements easy to add.